Project - 4

Pseudocode for a Secure Online Communication System Using TLS

Introduction

This project presents pseudocode for a client-server communication system that uses Transport Layer Security (TLS) to provide confidentiality, integrity, and authentication for data in transit. The design also includes controls that specifically defend against man-in-the-middle (MITM) attacks, in which an adversary intercepts, relays, or alters traffic between two endpoints. Security is applied at the design level rather than added after implementation, consistent with secure design principles (Richardson & Thies, 2013).

System Overview

The system consists of two components:

- **Server:** Holds a CA-signed X.509 certificate and private key, enforces TLS 1.3 (with TLS 1.2 as a restricted fallback), and optionally requires client certificates (mutual TLS).
- **Client:** Validates the server's certificate chain, hostname, validity period, and revocation status, and enforces certificate pinning before transmitting any application data.

Both sides fail closed: any validation or handshake error terminates the connection without sending data.

Pseudocode

Configuration Constants

```
CONSTANT MIN_TLS_VERSION      = TLS_1_2
CONSTANT PREFERRED_TLS        = TLS_1_3
CONSTANT ALLOWED_CIPHERS      = [
    TLS_AES_256_GCM_SHA384,                    // TLS 1.3
```

```
    TLS_CHACHA20_POLY1305_SHA256,               // TLS 1.3
    TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384,    // TLS 1.2, forward
secrecy
    TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384       // TLS 1.2, forward
secrecy
]
CONSTANT TRUSTED_CA_STORE     = LOAD("trusted_root_cas.pem")
CONSTANT PINNED_KEY_HASHES    = ["sha256/AbC123...",
"sha256/BackupKey456..."]
CONSTANT SERVER_HOSTNAME      = "secure.example.com"
CONSTANT SESSION_TIMEOUT      = 15 MINUTES
```

Server Setup

```
FUNCTION StartSecureServer(port)
    context = NEW TLSContext(SERVER_MODE)
    context.SetMinVersion(MIN_TLS_VERSION)
    context.SetCipherSuites(ALLOWED_CIPHERS)
    context.DisableCompression()                 // mitigates CRIME
    context.DisableRenegotiation()
    context.LoadCertificate("server_cert.pem")
    context.LoadPrivateKey(SECURE_KEYSTORE.Get("server_key"))
    context.EnableOCSPStapling()

    IF MUTUAL_TLS_REQUIRED THEN
        context.RequireClientCertificate(TRUSTED_CA_STORE)
    END IF

    listener = OPEN_SOCKET(port)
    LOOP
        rawConn = listener.Accept()
        TRY
            tlsConn = context.WrapSocket(rawConn)
            tlsConn.PerformHandshake()
            IF tlsConn.NegotiatedVersion < MIN_TLS_VERSION THEN
                THROW HandshakeError("Protocol downgrade detected")
            END IF
            LogEvent("Handshake OK", tlsConn.PeerAddress,
tlsConn.Cipher)
```

```
            SPAWN HandleClient(tlsConn)
        CATCH error
            LogSecurityEvent("Handshake failed", rawConn.PeerAddress,
error)
            rawConn.Close()
        END TRY
    END LOOP
END FUNCTION
```

Client Connection and Certificate Validation

```
FUNCTION ConnectSecurely(host, port)
    context = NEW TLSContext(CLIENT_MODE)
    context.SetMinVersion(MIN_TLS_VERSION)
    context.SetCipherSuites(ALLOWED_CIPHERS)
    context.LoadTrustStore(TRUSTED_CA_STORE)
    context.SetVerifyMode(VERIFY_REQUIRED)       // never disable
verification
    context.SetSNI(host)

    rawConn = OPEN_TCP(host, port)
    tlsConn = context.WrapSocket(rawConn)

    TRY
        tlsConn.PerformHandshake()
        cert = tlsConn.GetPeerCertificateChain()

        IF NOT ValidateCertificate(cert, host) THEN
            THROW SecurityError("Certificate validation failed")
        END IF
        IF NOT VerifyPin(cert.Leaf) THEN
            THROW SecurityError("Certificate pin mismatch: possible
MITM")
        END IF

        RETURN tlsConn
    CATCH error
        LogSecurityEvent("Connection aborted", host, error)
        tlsConn.Close()
```

```
        RETURN NULL                                  // fail closed
    END TRY
END FUNCTION


FUNCTION ValidateCertificate(chain, expectedHost)
    IF NOT chain.VerifySignatures(TRUSTED_CA_STORE) THEN RETURN FALSE
    IF CURRENT_TIME < chain.Leaf.NotBefore
       OR CURRENT_TIME > chain.Leaf.NotAfter THEN RETURN FALSE
    IF NOT MatchHostname(chain.Leaf.SubjectAltNames, expectedHost)
THEN
        RETURN FALSE
    END IF
    IF chain.Leaf.KeyUsage DOES NOT INCLUDE "serverAuth" THEN RETURN
FALSE

    revocation = CheckOCSP(chain.Leaf)           // or stapled
response
    IF revocation.Status != GOOD THEN RETURN FALSE

    RETURN TRUE
END FUNCTION


FUNCTION VerifyPin(leafCert)
    spkiHash = "sha256/" +
BASE64(SHA256(leafCert.SubjectPublicKeyInfo))

    RETURN spkiHash IN PINNED_KEY_HASHES
END FUNCTION
```

Secure Message Exchange

```
FUNCTION SendMessage(tlsConn, userSession, payload)
    IF userSession.IsExpired(SESSION_TIMEOUT) THEN
        THROW AuthError("Session expired")
    END IF

    message = {
```

```
        "session_id": userSession.Id,
        "nonce":      SECURE_RANDOM(16),          // replay protection
        "timestamp":  CURRENT_UTC_TIME,
        "body":       Sanitize(payload)
    }
    tlsConn.Write(SERIALIZE(message))            // encrypted + AEAD-
authenticated by TLS
END FUNCTION


FUNCTION HandleClient(tlsConn)
    WHILE tlsConn.IsOpen
        data = tlsConn.Read()
        message = DESERIALIZE_SAFE(data)

        IF NOT SessionStore.IsValid(message.session_id) THEN
            Reject(tlsConn, "Invalid session")
        ELSE IF NonceCache.Contains(message.nonce)
             OR ABS(CURRENT_UTC_TIME - message.timestamp) > 30 SECONDS
THEN
            LogSecurityEvent("Replay attempt", tlsConn.PeerAddress)
            Reject(tlsConn, "Replay detected")
        ELSE
            NonceCache.Add(message.nonce)
            ProcessRequest(ValidateInput(message.body))
        END IF
    END WHILE
    tlsConn.Close()
END FUNCTION
```

HTTP Security Headers (Web Deployment)

```
FUNCTION ApplySecurityHeaders(response)
    response.Header("Strict-Transport-Security",
                    "max-age=31536000; includeSubDomains; preload")
    response.Header("Content-Security-Policy", "default-src 'self'")
    response.Cookie.SetFlags(Secure, HttpOnly, SameSite=Strict)
```

```
END FUNCTION

FUNCTION HandlePlainHTTP(request)
    REDIRECT 301 TO "https://" + SERVER_HOSTNAME + request.Path
END FUNCTION
```

Description of Security Controls

Secure Communication with TLS

- **Protocol enforcement:** The minimum version is set to TLS 1.2, with TLS 1.3 preferred. SSL 3.0, TLS 1.0, and TLS 1.1 are rejected because of known weaknesses such as POODLE and BEAST. A post-handshake version check aborts any connection that falls below the minimum.
- **Cipher suite restriction:** Only AEAD ciphers (AES-GCM and ChaCha20-Poly1305) with ephemeral Elliptic Curve Diffie-Hellman (ECDHE) key exchange are permitted. AEAD provides confidentiality and integrity in a single operation, and ECDHE provides perfect forward secrecy (PFS), so a future compromise of the server's private key cannot decrypt previously captured sessions.
- **Attack surface reduction:** TLS compression and renegotiation are disabled to mitigate CRIME and renegotiation injection attacks.
- **Key protection:** The server's private key is retrieved from a secure keystore (such as an HSM or secrets manager) rather than stored in plaintext alongside application code.

Protection Against MITM Attacks

- **Mandatory certificate validation:** The client verifies the full chain of trust to a trusted root CA, the certificate's validity window, its Extended Key Usage, and its revocation status through OCSP. Verification is never disabled, which is a common implementation flaw that exposes systems to interception.
- **Hostname verification:** The hostname is matched against the certificate's Subject Alternative Name (SAN) entries, preventing an attacker from presenting a valid certificate issued for a different domain.
- **Certificate pinning:** The client compares the SHA-256 hash of the server's Subject Public Key Info (SPKI) against pre-approved values. Even if an attacker obtains a fraudulent certificate from a compromised or rogue CA, the pin check fails. A backup pin is included to allow key rotation without breaking clients.

- **Mutual TLS (optional):** When enabled, the server also authenticates the client by certificate, ensuring that both endpoints are verified identities.
- **HSTS and HTTPS redirection:** HTTP Strict Transport Security instructs browsers to use only HTTPS for the domain, blocking SSL-stripping attacks that downgrade connections to plaintext. Plain HTTP requests are permanently redirected.
- **Replay protection:** Each application message carries a cryptographically random nonce and timestamp. The server rejects duplicate nonces and stale timestamps, preventing an attacker from capturing and resending valid messages.
- **Fail-closed behavior and logging:** Any handshake, validation, or pin failure terminates the connection before data is exchanged, and the event is logged for monitoring and incident response.

Conclusion

The pseudocode establishes encrypted, authenticated channels using TLS 1.3 with forward-secret AEAD cipher suites, and layers MITM defenses on top: strict certificate and hostname validation, revocation checking, certificate pinning, HSTS, optional mutual TLS, and replay protection. By failing closed on any trust violation, the system ensures that data is never transmitted over a channel whose integrity cannot be verified.

References

Richardson, T., & Thies, C. N. (2013). *Secure software design*. Jones & Bartlett Learning.

Anthropic. (2026). *Claude* (Opus 5.5 version) [Large language model]. https://claude.ai